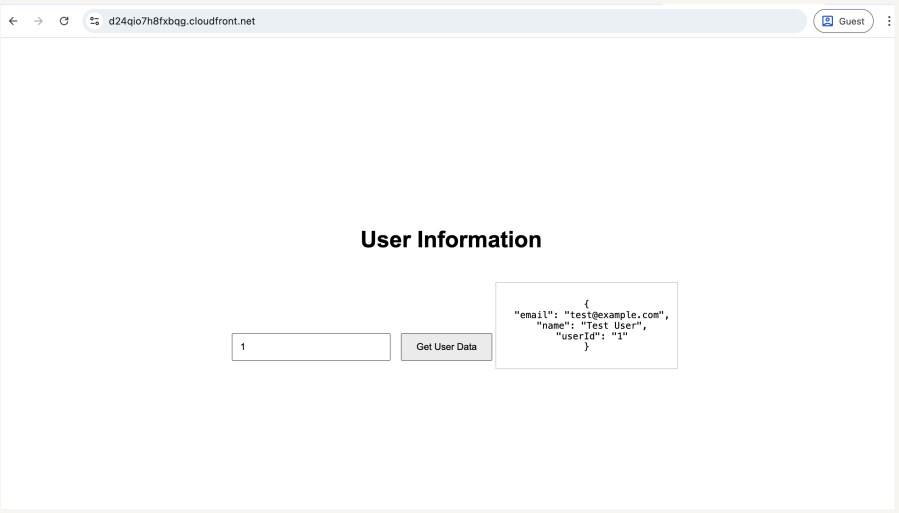
nextwork.org

Build a Three-Tier Web App

Kehinde Abiuwa



Kehinde Abiuwa
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate how to build a full three-tier web app on AWS—host the frontend in S3, deliver it globally with CloudFront, power the backend with Lambda behind API Gateway, and persist data in DynamoDB—then connect all these services into one working application. I'm doing this project to learn how the presentation, logic, and data tiers fit together on AWS: creating an S3 bucket, setting up a CloudFront distribution, writing and deploying a Lambda function, defining REST endpoints with API Gateway, modeling/fetching data in DynamoDB, and wiring everything up securely using least-privilege IAM.

Tools and concepts

Services I used were Amazon S3, Amazon CloudFront, AWS Lambda, Amazon API Gateway, and Amazon DynamoDB. Key concepts I learnt include Lambda functions, serverless scaling, API Gateway REST endpoints, CORS (and why both API Gateway and Lambda responses may need CORS headers), CloudFront caching & invalidations, S3 static site hosting, DynamoDB data modeling (partition key userId) with DocumentClient GetItem, and least-privilege IAM roles/policies to securely connect the three tiers.

Project reflection

This project took me approximately 1 hour and 12 minutes. The most challenging part was debugging why the CloudFront page couldn't call the API—first a placeholder API URL causing 403/"Unexpected token <" errors, then CORS and caching.

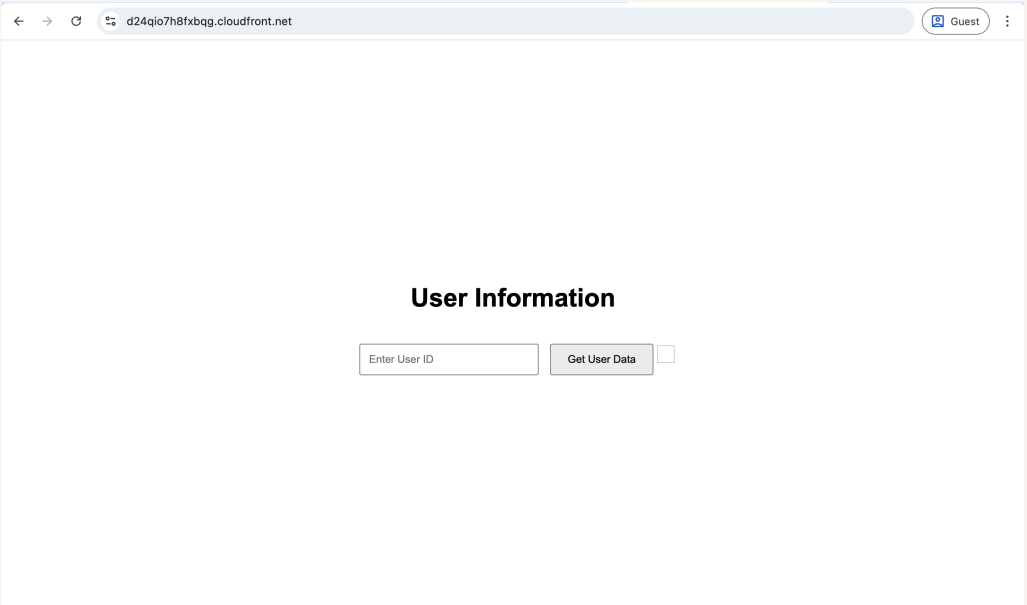
I fixed it by updating script.js with the real Invoke URL, invalidating CloudFront, and enabling CORS in both API Gateway (and Lambda responses). It was most rewarding to reload the CloudFront site, enter 1, click Get User Data, and see a 200 OK with the DynamoDB user JSON—proof the whole three-tier stack works end-to-end.

I chose to do this project today because... Something that would make learning with NextWork even better is...

Presentation tier

For the presentation tier, I will set up an S3 bucket, upload a simple index.html, and front it with a CloudFront distribution because this top layer is responsible for what users see and needs fast, global delivery of my website's files.

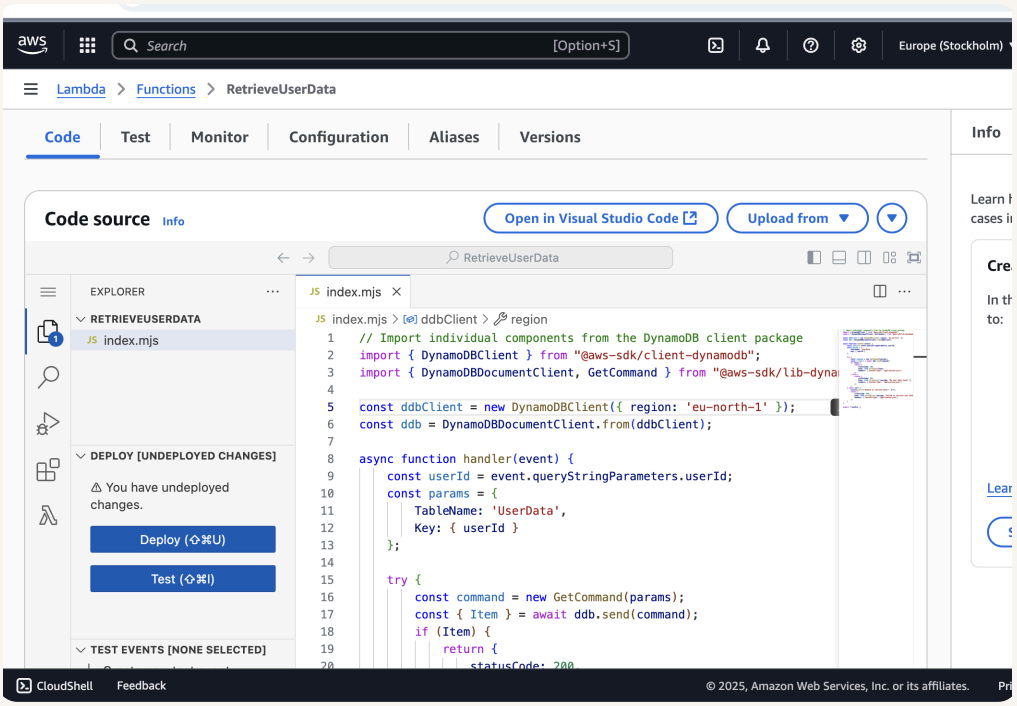
I accessed my delivered website by opening the CloudFront distribution URL (the *.cloudfront.net domain shown in the CloudFront console) in my browser to load my index.html.



Logic tier

For the logic tier, I will set up an AWS Lambda function and expose it through API Gateway because this middle layer handles the app's brain—processing requests and fetching data from DynamoDB for the frontend.

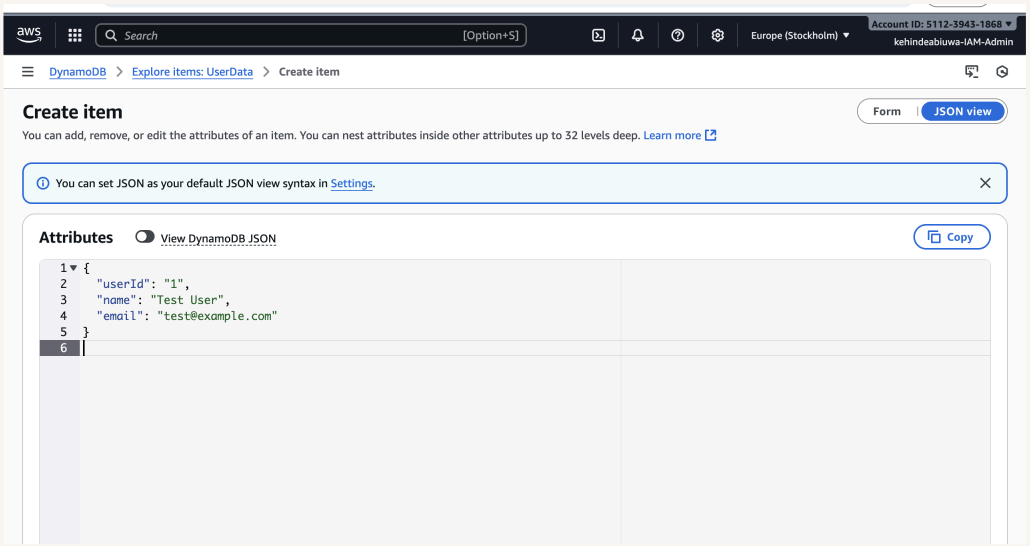
The Lambda function retrieves data by reading a userId from the request, then using the AWS SDK's DynamoDB DocumentClient to run a GetItem against the UserData table (partition key: userId). If the item exists, it returns the user data as JSON; if not, it returns a "not found" message. Errors are logged, and access is allowed via the function's IAM role with dynamodb:GetItem permission.


aws
Search
[Option+S]
Europe (Stockholm)
Lambda
Functions
RetrieveUserData
Code
Test
Monitor
Configuration
Aliases
Versions
Info
Code source
Info
Open in Visual Studio Code
Upload from
RetrieveUserData
EXPLORER
RETRIEVEUSERDATA
index.mjs
DEPLOY [UNDEPLOYED CHANGES]
You have undeployed changes.
Deploy (⇧⌘U)
Test (⇧⌘I)
TEST EVENTS [NONE SELECTED]
index.mjs
ddbClient
region
// Import individual components from the DynamoDB client package
import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dyna
const ddbClient = new DynamoDBClient({ region: 'eu-north-1' });
const ddb = DynamoDBDocumentClient.from(ddbClient);
async function handler(event) {
const userId = event.queryStringParameters.userId;
const params = {
TableName: 'UserData',
Key: { userId }
};
try {
const command = new GetCommand(params);
const { Item } = await ddb.send(command);
if (Item) {
return {
statusCode: 200
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.

Data tier

For the data tier, I will set up an Amazon DynamoDB table and add sample user records because this layer stores the application's data that my Lambda + API Gateway will read and return to the website.

The partition key for my DynamoDB table is userId which means each user record is uniquely identified and quickly retrievable by its userId



Logic and Data tier

Once all three layers of my three-tier architecture are set up, the next step is to integrate them by wiring the frontend to the API, because this proves the presentation tier can talk to the logic tier, which then reads from the data tier.

To test my API, I opened the API Gateway Invoke URL in the browser and sent a GET request with the query string ?userId=1 (e.g., https://<api-id>.execute-api.<region>.amazonaws.com/prod/users?userId=1). The results were a 200 OK with the JSON payload: {"email":"test@example.com","name":"Test User","userId":"1"} which confirms the API Gateway → Lambda → DynamoDB path is working end-to-end.

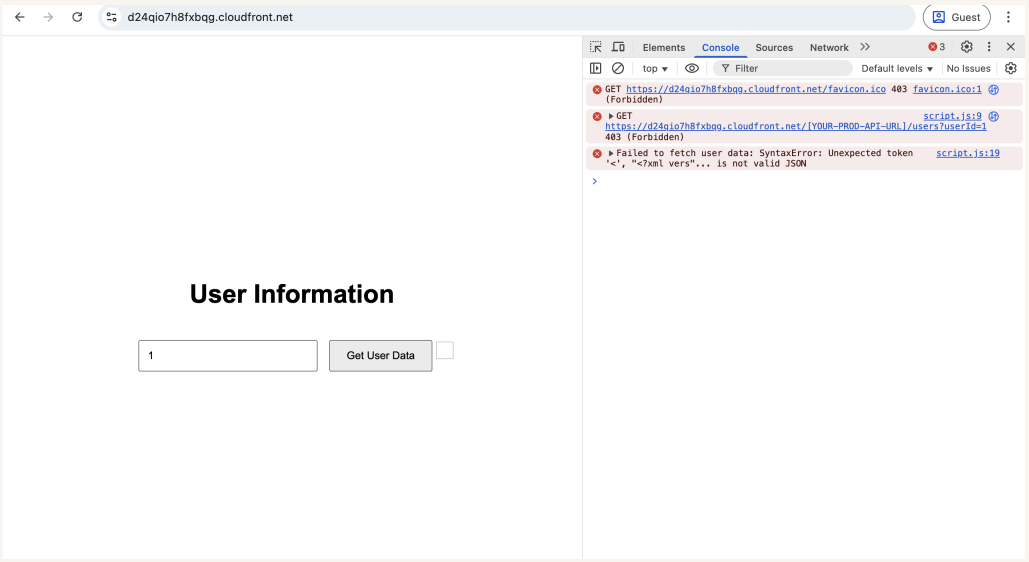

i9m9481v17.execute-api.eu-north-1.amazonaws.com/prod/users?userId=1
Pretty print
{"email":"test@example.com","name":"Test User","userId":"1"}

Console Errors

The error in my distributed site was because script.js still used the placeholder https://[YOUR-PROD-API-URL]/users?userId=1, so the browser tried to fetch from a non-real URL and threw a “Failed to parse URL”/fetch error (as shown in the console trace from script.js:9).

To resolve the error, I updated script.js by inserting the correct invoke url i got from my api gateway. I then reuploaded it into S3 because CloudFront serves my site from the S3 origin.

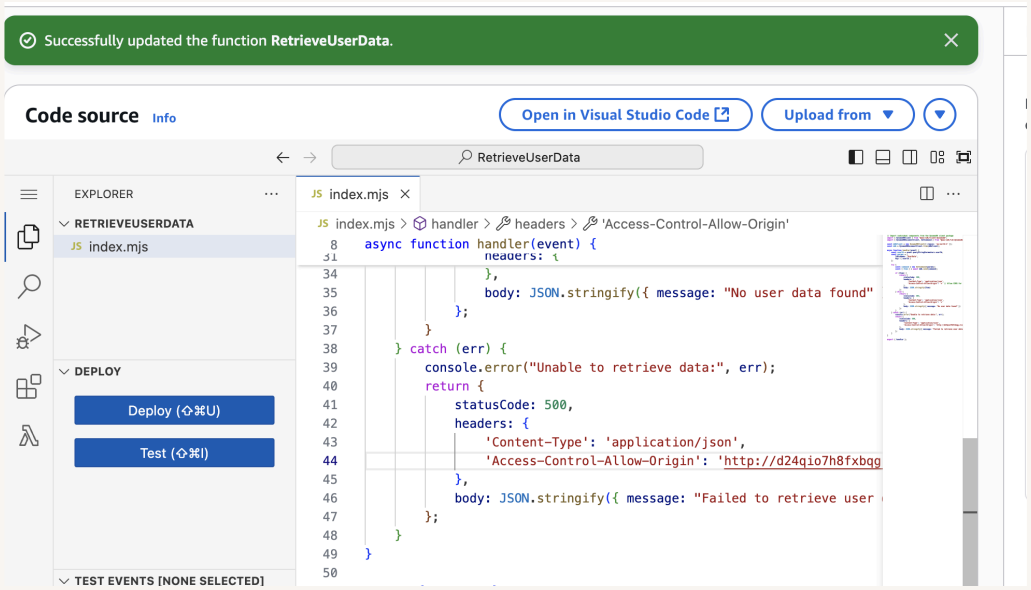
I ran into a second error after updating script.js. This was an error with CORS (Cross-Origin Resource Sharing) because because my browser was loading the site from my CloudFront domain but calling the API on a different origin (API Gateway), and the API wasn’t returning the required CORS headers (e.g., Access-Control-Allow-Origin). By default, browsers block cross-origin requests unless the backend explicitly allows the frontend’s domain


d24qio7h8fxbqg.cloudfront.net
Guest
Elements
Console
Sources
Network
top
Filter
Default levels
No Issues
GET https://d24qio7h8fxbqg.cloudfront.net/favicon.ico 403 favicon.ico:1
(Forbidden)
GET script.js:9
https://d24qio7h8fxbqg.cloudfront.net/[YOUR-PROD-API-URL]/users?userId=1
403 (Forbidden)
Failed to fetch user data: SyntaxError: Unexpected token script.js:19
'<', "<?xml vers"... is not valid JSON
User Information
1
Get User Data

Resolving CORS Errors

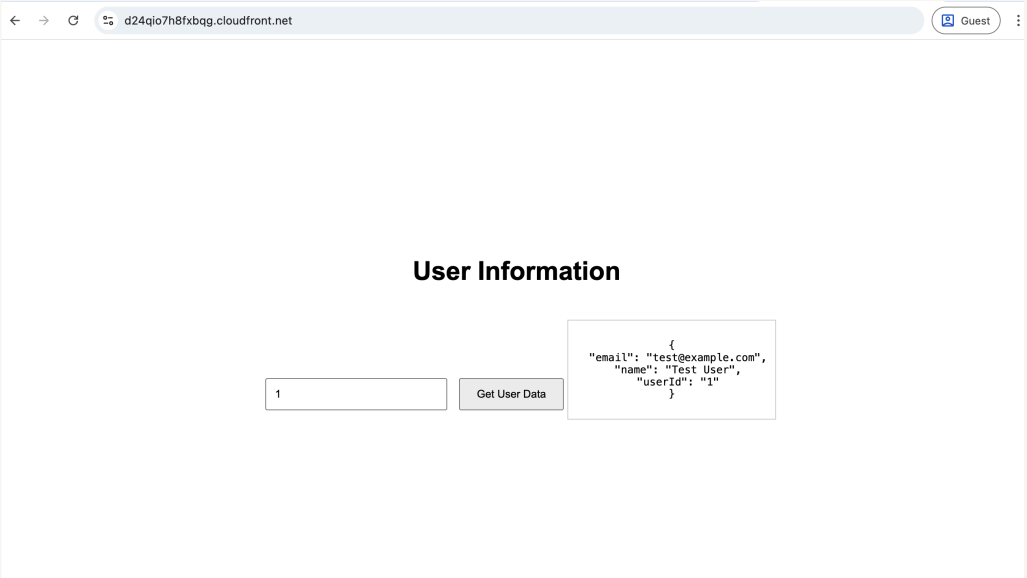
To resolve the CORS error, I first enabled CORS on my API Gateway resource/method, adding my CloudFront domain to Access-Control-Allow-Origin. I kept GET as the allowed method and included the standard headers.

I also updated my Lambda function because the browser needs CORS headers on the actual response from Lambda (with proxy integration), otherwise the fetch is still blocked even if CORS is enabled in API Gateway. The changes I made were: Return a proxy-style response with statusCode, headers, and body. Add CORS headers on every response: Access-Control-Allow-Origin: https://<my-cloudfront-domain> (or * for quick tests), Access-Control-Allow-Methods: GET,OPTIONS, Access-Control-Allow-Headers: Content-Type,Authorization,...


Successfully updated the function RetrieveUserData.
Code source Info
Open in Visual Studio Code
Upload from
RetrieveUserData
EXPLORER
RETRIEVEUSERDATA
index.mjs
DEPLOY
Deploy (⇧⌘U)
Test (⇧⌘I)
TEST EVENTS [NONE SELECTED]
index.mjs
index.mjs > handler > headers > 'Access-Control-Allow-Origin'
async function handler(event) {
body: JSON.stringify({ message: "No user data found"
console.error("Unable to retrieve data:", err);
return {
statusCode: 500,
headers: {
'Content-Type': 'application/json',
'Access-Control-Allow-Origin': 'http://d24qio7h8fxbqg
body: JSON.stringify({ message: "Failed to retrieve user

Fixed Solution

I verified the fix by reloading my CloudFront URL, entering 1, and clicking Get User Data. The page returned the user JSON from DynamoDB, and the Network tab showed a 200 OK from the API Gateway Invoke URL



The place to learn & showcase your skills

Check out nextwork.org for more projects